

Comparison between MMSE, Ridge Regression and Support Vector Regression

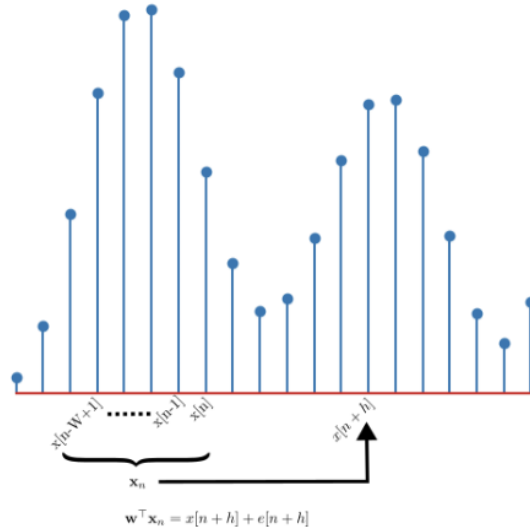
Model of the time series and data structure

In the present homework, a time series has to be predicted from past samples with a linear regression algorithm. The time series is defined as the discrete function $x[n], 0 \leq n \leq N - 1$, consisting of N samples with a high degree of correlation. Therefore, the value of a sample $x[n]$ depends on past samples. We model the sequence as

$$f(x[n - W + 1], \dots, x[n]) + b = x[n + h] + e[n + h]$$

Where b is a bias term. This is, we hypothesize that the value of the sequence at instant $n + h$ is a function of the values of the sequence at past instants from instant $n - W + 1$ to instant n . The linear function that is constructed to model this hypothesis is then

$$x[n + h] + e[n + h] = \sum_{i=0}^{W-1} w_i x[n - i] + b,$$



The previous equation can be written as a dot product between the two vectors

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{W-1} \end{bmatrix}, \quad \mathbf{x}_n = \begin{bmatrix} x_n \\ x_{n-1} \\ \vdots \\ x_{n-W+1} \end{bmatrix},$$

this is

$$x[n + h] + e[n + h] = \mathbf{w}^T \mathbf{x}_n + b,$$

which has the form of a linear regressor.

In order to find the optimal parameters to predict instant $n + h$ (called prediction at horizon h) from the observations in the window of length W from instants $n - W + 1$ to n , we are given a training sequence x_{tr} with N samples, and validation and test sequences x_{val} and x_{test} , also with N samples.

In order to train the regression model, we first need to construct a training data set with pairs $\mathbf{x}_n, y[n]$, where $y[n] = x[n + h]$. The matrix containing all predictors $\mathbf{x}_n$ is

$$\mathbf{X}_{tr} = [\mathbf{x}_{tr,W-1}, \dots, \mathbf{x}_{tr,N}]$$

and the vector containing all regressors is $\mathbf{y}_{tr} = [y_{tr,1}, y_{tr,2}, \dots, y_{tr,N-h}]$

They are constructed as

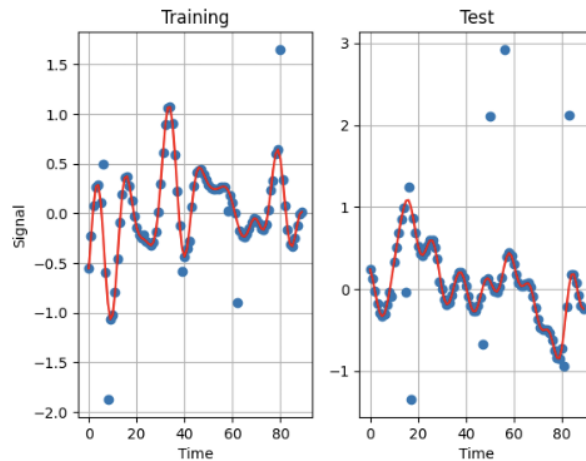
$$\mathbf{y}_{tr} = [x[W - 1 + h] \quad x[W + h] \quad \cdots \quad x[N]]$$

$$\mathbf{X}_{tr} = \begin{bmatrix} x[W - 1] & x[W] & \cdots & x[N - h] \\ x[W] & x[W + 1] & \cdots & x[N - h + 1] \\ \vdots & \vdots & \ddots & \vdots \\ x[1] & x[2] & \cdots & x[N - h - W + 2] \\ x[0] & x[1] & \cdots & x[N - h - W + 1] \end{bmatrix}$$

The sequence window is often called a buffer, and the operation of constructing this matrix is then called buffering. The provided [Google Colab](#) Links to an external site. notebook contains a buffering function and the code necessary to produce the corresponding vector of regressors for arbitrary values of W and h . **The student is required to study and identify the previous theory in the provided notebook in order to be able to complete the homework.**

Noise and outlier model

The function that returns the time series contains a parameter to specify the Gaussian noise standard deviation. Also, there is a function that adds an outlier to each sample with probability p . The outlier is drawn from a Gaussian distribution with zero mean and standard deviation a .



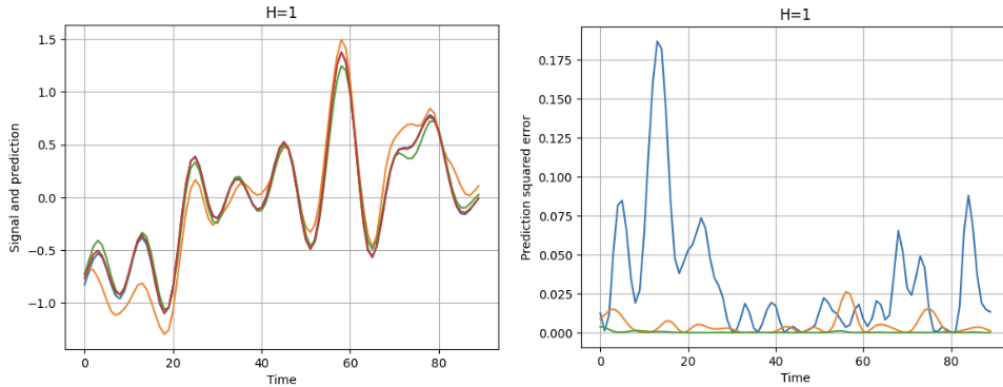
The image above shows the comparison between the "clean" signal (in red) and the same signal contaminated with outliers. We will use a signal with outliers to train the regressors.

Homework

5.1 Comparison between MMSE, RR and SVM.

- Construct two different datasets for training and test, each with 100 samples and no Gaussian noise added. The predictor window has $W = 10$ samples and the prediction horizon is $H = 1$.
- Train and test an MMSE regressor, whose solution is $\mathbf{w} = (\mathbf{X}\mathbf{X}^\top)^{-1}\mathbf{X}\mathbf{y}$, a Ridge Regression one, with solution $\mathbf{w} = (\mathbf{X}\mathbf{X}^\top + \gamma\mathbf{I})^{-1}\mathbf{X}\mathbf{y}$ and $\gamma = 0.2$, and a SVR with $C = 10$ and $\varepsilon = 0.01$. The training regressors will be added outliers with probability $p = 0.1$ and standard deviation $a = 1$.
- Plot the prediction output on the test set for the three prediction methods and compare them with the clean test signal (without outliers).
- Plot the squared error between the clean test signal and the predictions as a function of time.
- Provide an explanation of the results.

The graphs should have an aspect similar to the one in the following figures. Do not forget to label the graphs.



Solution 5.1

A. Dataset construction

We generated two independent sequences $x_{\text{train}}[n]$ and $x_{\text{test}}[n]$ of length $N = 100$ using the provided time-series generator with $\sigma = 0$ (no Gaussian noise). For each sequence, we constructed regression windows of length $W = 10$ and a prediction horizon $H = 1$. This yields

$$\mathbf{X}_{\text{train}} \in \mathbb{R}^{90 \times 10}, \quad \mathbf{y}_{\text{train, clean}} \in \mathbb{R}^{90},$$

and similarly for the test set $(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test, clean}})$.

B. MMSE, Ridge and SVR training

We corrupted the training targets with outliers using probability $p = 0.1$ and standard deviation $a = 1$:

$$\mathbf{y}_{\text{train}} = \mathbf{y}_{\text{train, clean}} + \text{outliers}.$$

Using $\mathbf{X} = \mathbf{X}_{\text{train}}^T$ and $\mathbf{y} = \mathbf{y}_{\text{train}}$, the MMSE and Ridge solutions are

$$\mathbf{w}_{\text{MMSE}} = (\mathbf{X}\mathbf{X}^T)^{-1}\mathbf{X}\mathbf{y}, \quad \mathbf{w}_{\text{Ridge}} = (\mathbf{X}\mathbf{X}^T + \gamma\mathbf{I})^{-1}\mathbf{X}\mathbf{y},$$

with $\gamma = 0.2$. The SVR model is trained with a linear kernel, $C = 10$ and $\varepsilon = 0.01$.

C. Prediction plots

Using the clean test inputs $\mathbf{X}_{\text{test}}$, we computed the predictions

$$\hat{\mathbf{y}}_{\text{MMSE}} = \mathbf{X}_{\text{test}}\mathbf{w}_{\text{MMSE}}, \quad \hat{\mathbf{y}}_{\text{Ridge}} = \mathbf{X}_{\text{test}}\mathbf{w}_{\text{Ridge}}, \quad \hat{\mathbf{y}}_{\text{SVR}} = \text{SVR}(\mathbf{X}_{\text{test}}).$$

Figure 1 shows the clean test signal and the three predictions.

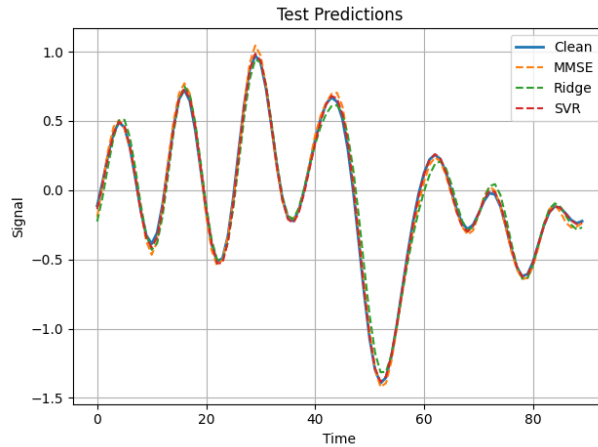


Figure 1: Clean test signal and predictions from MMSE, Ridge, and SVR.

D. Squared error plots

We computed the pointwise squared error

$$e_{\text{method}}[n] = (y_{\text{test, clean}}[n] - \hat{y}_{\text{method}}[n])^2$$

for each method. Figure 2 shows the squared error over time.

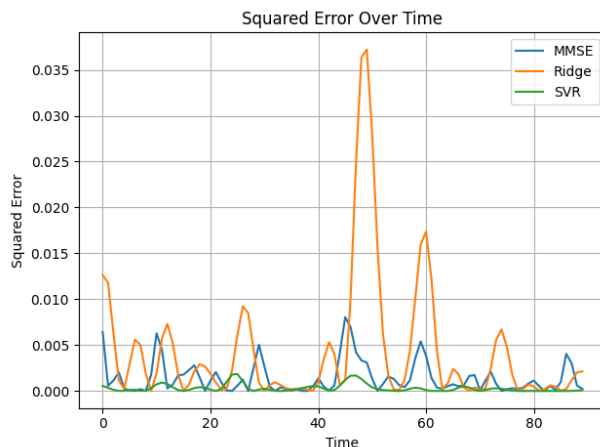


Figure 2: Squared error over time for MMSE, Ridge, and SVR.

E. Discussion of the results

All three methods track the overall shape of the clean test signal, but their robustness to outliers differs. MMSE, which minimizes the mean squared error, is sensitive to large errors in the training targets and shows moderate error spikes. Ridge Regression shrinks the weights through the regularization term $\gamma\|\mathbf{w}\|^2$; in this setting it tends to underfit the signal and exhibits the largest squared errors.

SVR, trained with an ε -insensitive loss, is much more robust to outliers in the labels. Small errors inside the ε -tube are ignored, and only samples with large residuals become support vectors. As a result, SVR achieves the smallest squared error over time and provides the closest approximation to the clean test signal.

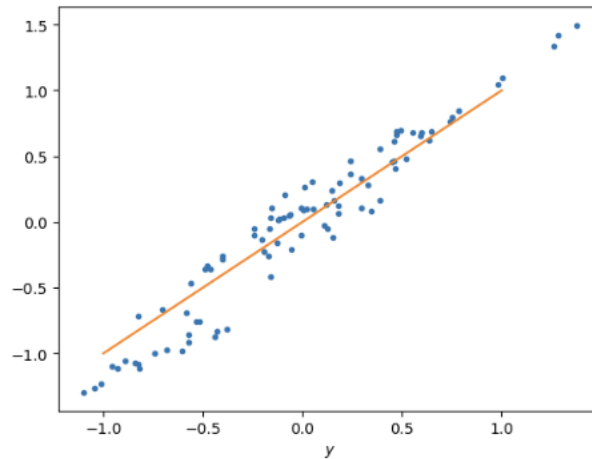
5.2 Performance measures using the correlation coefficient.

In order to assess the performance of a regressor, a possible measure of performance is the Pearson Correlation Coefficient, which is normalized to 1. Its expression is

$$\rho_{y\hat{y}} = \frac{\text{Cov}(y, \hat{y})}{\sigma_y \sigma_{\hat{y}}},$$

where y and $\hat{y}$ are the true and predicted signals respectively. If the prediction is perfect, then, the covariance will be equal to the variance of the signal and the coefficient will be 1. If the prediction does not work at all, the signals will be uncorrelated, and the covariance will be zero.

- A. Compute the correlation coefficient of the prediction for different horizons between 1 and 10 between the MMSE solution and the SRV with the parameters of 5.1. The training regressor will have the same outlier probability and standard deviation as in 5.1. You may need to repeat the experiment several times and average the results.. The correlation coefficient must be computed with the test regressor without outliers. Explain the results.
- B. Make a graph that shows the correlation between the predictions and the true values by plotting each prediction as a function of the true value, and the identity line superimposed. Do it for $H=1$ and 5 and for both algorithms. They should have an aspect similar to the one of the figure



- C. Explain the graphs.
- D. Generate a validation set of 100 samples. For $H=1$ and $H=2$, construct a crossvalidation loop to select the best possible value of C and ε for the SVR as a function of the correlation coefficient. You can use the following code to generate the values of both parameters

```

1         epsilons = np.logspace(-2, 0, max_epsilons)
2         cs       = np.logspace(-2, 2, max_cs)
3

```

- E. Report the optimum values, plot a contour plot of the correlation coefficient as a function of both parameters.
- F. Plot a test prediction result for the optimum for both horizons. Comment ther result.

Link to Google colab code <https://colab.research.google.com/drive/14sskFGEKkjr1N7Zq3oHL1NjtsdjTL6zh?usp=>

Solution 5.2

A. Correlation coefficient vs. prediction horizon

For each $H \in \{1, \dots, 10\}$ we generated new training and test sequences, added outliers to the training targets ($p = 0.1$, $a = 1$), and trained an MMSE regressor and an SVR with $(C = 10, \varepsilon = 0.01)$. For each horizon we computed the Pearson correlation coefficient between the clean test signal and the model prediction, averaging over 20 trials.

The averaged correlation curves in Fig. 3 show that both models perform almost perfectly for small horizons, but SVR degrades much more quickly as H increases. The MMSE model stays more stable for longer horizons.

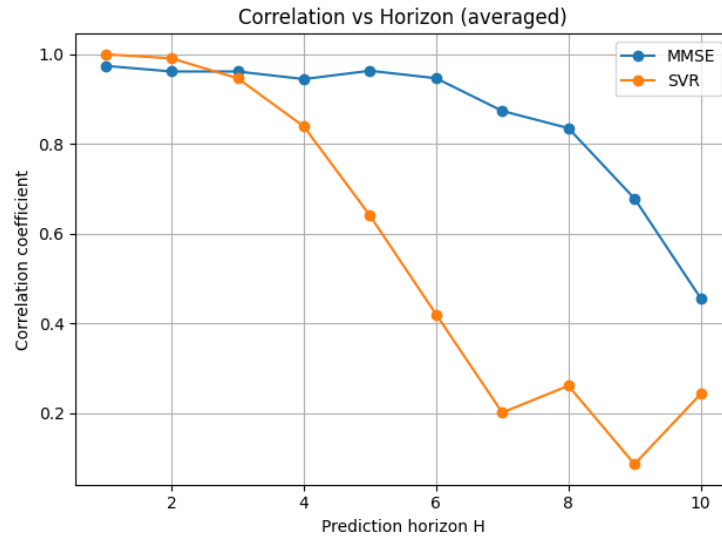


Figure 3: Average correlation coefficient for MMSE and SVR for horizons $H = 1$ – 10 .

B. Scatter plots of prediction vs. true values

To visualize how each regressor behaves at different horizons, we generated scatter plots for $H = 1$ and $H = 5$. The single combined figure (MMSE and SVR for both horizons) is shown in Fig. 4.

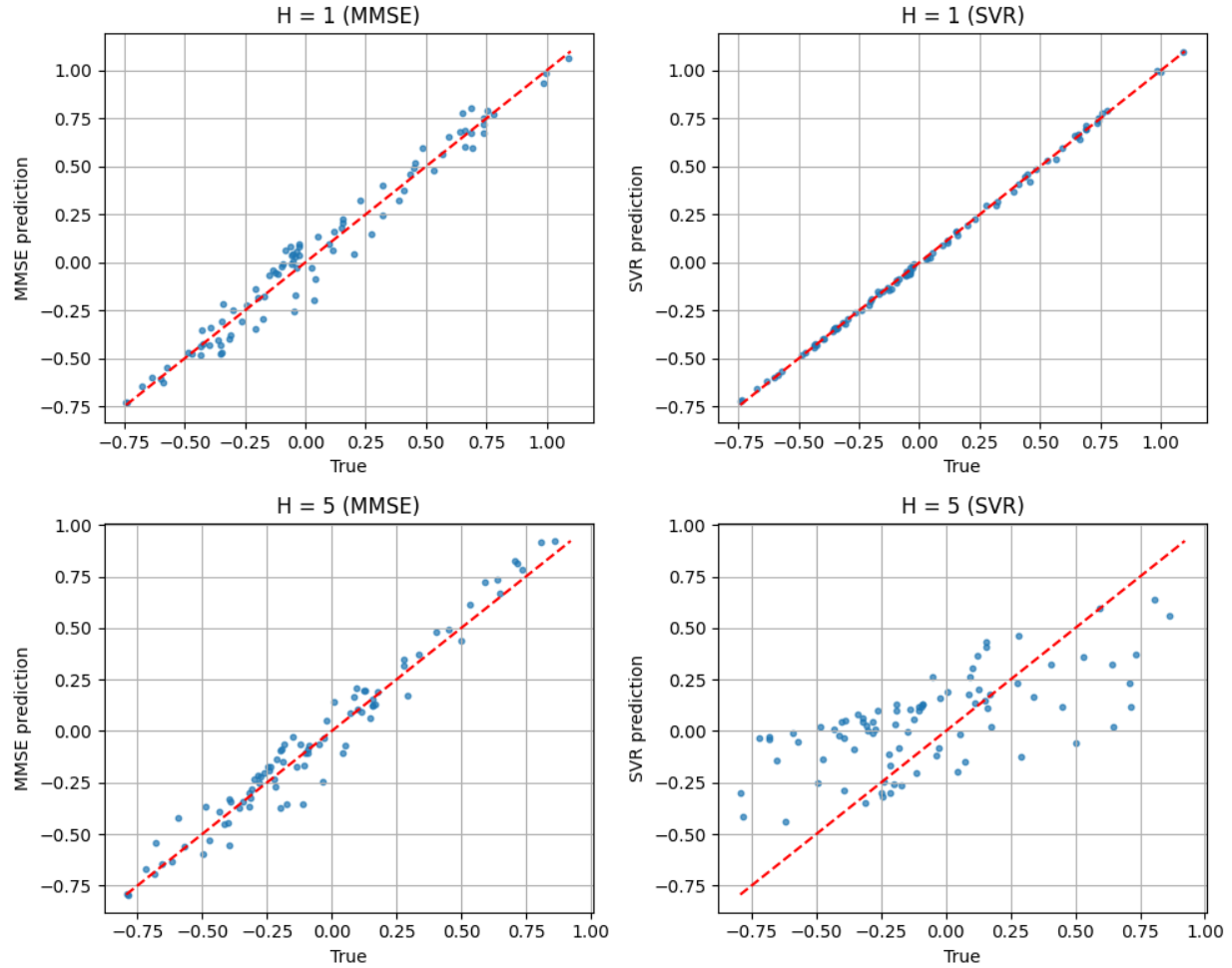


Figure 4: Scatter plots of prediction vs. true values for MMSE and SVR at $H = 1$ and $H = 5$. The identity line is shown in red.

C. Explanation of the scatter plots

At $H = 1$, both methods land nearly on the identity line, with SVR being almost perfectly aligned. At $H = 5$, MMSE still roughly tracks the structure of the true signal but shows more spread. SVR, on the other hand, shows large dispersion and tends to underestimate larger values. This aligns with the horizon-dependent correlations in part A.

D. Cross-validation over C and ε

We performed a grid search over

```

1     epsilons = np.logspace(-2, 0, max_epsilons)
2     cs       = np.logspace(-2, 2, max_cs)
3

```

using a clean validation sequence. The correlation maps for $H = 1$ and $H = 2$ are shown together in Fig. 5. In both cases, the best performance occurs at a small ε and large C , with the optimal values

$$\varepsilon^* = 0.01, \quad C^* = 100.$$

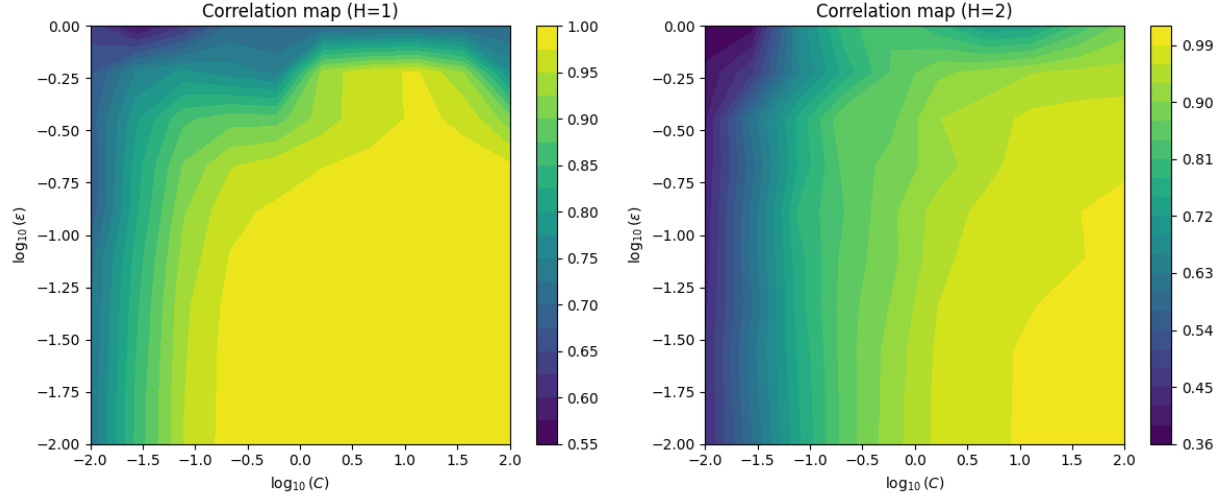


Figure 5: Validation correlation maps for (C, ϵ) for $H = 1$ (left) and $H = 2$ (right).

E. Test performance with optimal parameters

Using the optimal hyperparameters above, we retrained the SVR and evaluated it on the clean test signal. The test correlations were

Horizon H	ϵ^*	Test Correlation ρ
1	0.01	0.9999
2	0.01	0.9981

The corresponding predictions (side-by-side) are shown in Fig. 6.

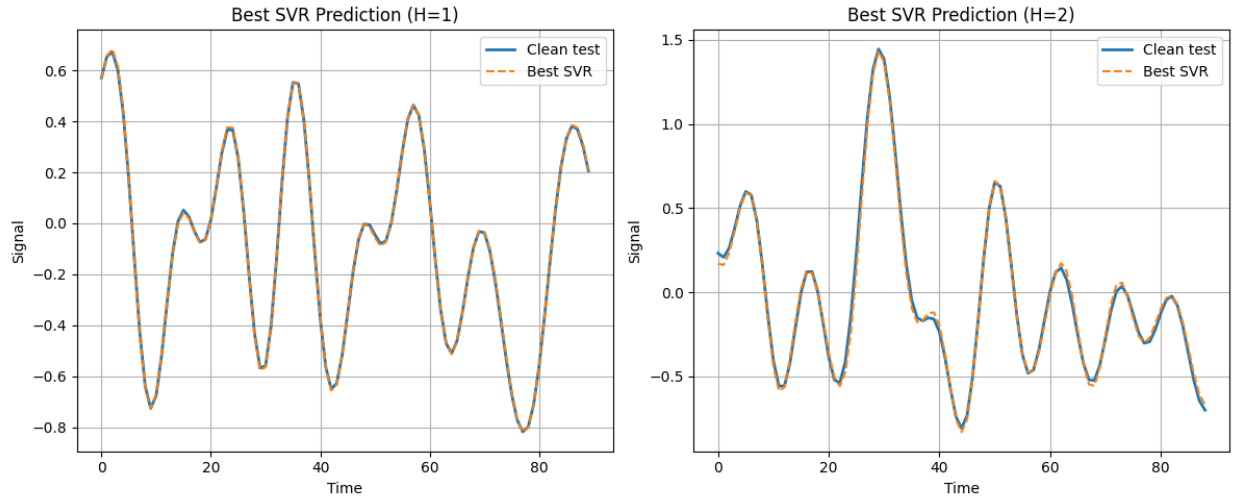


Figure 6: Best SVR predictions for $H = 1$ (left) and $H = 2$ (right).

F. Comment on the results

SVR performs extremely well for short-term prediction when C is large and ε is small—essentially matching the clean test signal at $H = 1$. However, as the horizon increases, SVR becomes more sensitive to outliers and the inherent difficulty of multi-step prediction, while the MMSE regressor degrades much more gradually. This makes MMSE more reliable for longer-term forecasts in this setting.

```

1 from scipy import signal
2 import matplotlib.pyplot as plt
3 from sklearn.svm import SVR
4 import numpy as np
5 import os
6
7 os.makedirs("plots", exist_ok=True)
8 np.random.seed(0)
9
10 def buffer(x, n, hop): # Generated with AI. This returns an array of buffered signal windows
11     """
12     Args:
13     x: Input array.
14     n: Length of each buffer.
15     hop: Hop size (The overlap is n-hop).
16
17     Returns:
18     A 2D array where each row is a buffer.
19     """
20     if len(x) < n:
21         raise ValueError("Input array length must be greater than or equal to the buffer length.")
22     num_buffers = (len(x) - n) // hop + 1
23     result = np.zeros((num_buffers, n))
24     for i in range(num_buffers):
25         start = i * hop
26         end = start + n
27         result[i, :] = x[start:end]
28     return result
29
30 def signal_gen(sigma, N):
31     T = 200
32     sos = signal.butter(10, 0.2, 'low', output='sos')
33     z = np.random.randn(N+T)
34     x = signal.sosfilt(sos, z)
35     x = x + sigma*np.random.randn(x.shape[0])
36     x = x[T:]
37     return x
38
39 def add_outliers(x, p, a):
40     x = x + a*(np.random.rand(x.shape[0]) > (1-p))*np.random.randn(x.shape[0])
41     return x
42
43 def build_datasets(N=100, W=10, H=1, sigma=0.0):
44     xtrain = signal_gen(sigma, N)
45     xval = signal_gen(sigma, N)
46     xtest = signal_gen(sigma, N)
47
48     Xtrain_full = buffer(xtrain, W, 1)
49     Xval_full = buffer(xval, W, 1)
50     Xtest_full = buffer(xtest, W, 1)
51
52     ytrain_clean = xtrain[W+H-1:]
53     yval_clean = xval[W+H-1:]

```

```

54 ytest_clean = xtest[W+H-1:]
55
56 Xtrain = Xtrain_full[0:-H]
57 Xval   = Xval_full[0:-H]
58 Xtest  = Xtest_full[0:-H]
59
60 return Xtrain, ytrain_clean, Xval, yval_clean, Xtest, ytest_clean
61
62 def corrcoef(y_true, y_pred):
63     return np.corrcoef(y_true, y_pred)[0, 1]
64
65 N = 100
66 W = 10
67 H = 1
68 sigma = 0.0
69
70 xtrain = signal_gen(sigma, N)
71 xtest  = signal_gen(sigma, N)
72
73 Xtrain_full = buffer(xtrain, W, 1)
74 ytrain_clean = xtrain[W+H-1:]
75 Xtrain = Xtrain_full[0:-H]
76
77 print("\n5.1A")
78 print("Xtrain shape:", Xtrain.shape)
79 print("ytrain_clean shape:", ytrain_clean.shape)
80
81 Xtest_full = buffer(xtest, W, 1)
82 ytest_clean = xtest[W+H-1:]
83 Xtest = Xtest_full[0:-H]
84
85 print("Xtest shape:", Xtest.shape)
86 print("ytest_clean shape:", ytest_clean.shape)
87
88 p = 0.1
89 a = 1.0
90 ytrain = add_outliers(ytrain_clean.copy(), p, a)
91
92 X = Xtrain.T
93 y = ytrain
94 XXT = X @ X.T
95 Xy = X @ y
96
97 w_mmse = np.linalg.solve(XXT, Xy)
98 w_ridge = np.linalg.solve(XXT + 0.2*np.eye(XXT.shape[0]), Xy)
99
100 svr_model = SVR(C=10.0, epsilon=0.01, kernel='linear')
101 svr_model.fit(Xtrain, ytrain)
102
103 yhat_mmse = Xtest @ w_mmse
104 yhat_ridge = Xtest @ w_ridge
105 yhat_svr = svr_model.predict(Xtest)
106
107 print("\n5.1B")

```

```
108 print("Models trained: MMSE, Ridge, SVR")
109
110 t = np.arange(len(ytest_clean))
111 plt.figure()
112 plt.plot(t, ytest_clean, label='Clean', linewidth=2)
113 plt.plot(t, yhat_mmse, '--', label='MMSE')
114 plt.plot(t, yhat_ridge, '--', label='Ridge')
115 plt.plot(t, yhat_svr, '--', label='SVR')
116 plt.xlabel('Time')
117 plt.ylabel('Signal')
118 plt.title('Test Predictions')
119 plt.grid()
120 plt.legend()
121 plt.tight_layout()
122 plt.savefig("plots/predictions.png")
123 plt.close()
124 print("5.1C: saved plots/predictions.png")
125
126 # 5.1D
127 se_mmse = (ytest_clean - yhat_mmse)**2
128 se_ridge = (ytest_clean - yhat_ridge)**2
129 se_svr = (ytest_clean - yhat_svr)**2
130
131 plt.figure()
132 plt.plot(t, se_mmse, label='MMSE')
133 plt.plot(t, se_ridge, label='Ridge')
134 plt.plot(t, se_svr, label='SVR')
135 plt.xlabel('Time')
136 plt.ylabel('Squared Error')
137 plt.title('Squared Error Over Time')
138 plt.grid()
139 plt.legend()
140 plt.tight_layout()
141 plt.savefig("plots/squared_error.png")
142 plt.close()
143 print("5.1D: saved plots/squared_error.png")
144
145 print("\n5.2A")
146
147 p = 0.1
148 a = 1.0
149 W = 10
150 sigma = 0.0
151 N = 100
152 num_trials = 20
153 Hs = np.arange(1, 11)
154
155 corr_mmse = np.zeros_like(Hs, dtype=float)
156 corr_svr = np.zeros_like(Hs, dtype=float)
157
158 for idx, H in enumerate(Hs):
159     cm = []
160     cs = []
161     for _ in range(num_trials):
```

```
162 Xtrain, ytrain_clean, _, _, Xtest, ytest_clean = build_datasets(N, W, H, sigma)
163 ytrain = add_outliers(ytrain_clean.copy(), p, a)
164
165 X = Xtrain.T
166 y = ytrain
167 XXT = X @ X.T
168 Xy = X @ y
169
170 w_mmse = np.linalg.solve(XXT, Xy)
171 yhat_mmse = Xtest @ w_mmse
172
173 svr = SVR(C=10.0, epsilon=0.01, kernel='linear')
174 svr.fit(Xtrain, ytrain)
175 yhat_svr = svr.predict(Xtest)
176
177 cm.append(corrcoef(ytest_clean, yhat_mmse))
178 cs.append(corrcoef(ytest_clean, yhat_svr))
179
180 corr_mmse[idx] = np.mean(cm)
181 corr_svr[idx] = np.mean(cs)
182
183 plt.figure()
184 plt.plot(Hs, corr_mmse, 'o-', label='MMSE')
185 plt.plot(Hs, corr_svr, 'o-', label='SVR')
186 plt.xlabel('Prediction horizon H')
187 plt.ylabel('Correlation coefficient')
188 plt.title('Correlation vs Horizon (averaged)')
189 plt.grid()
190 plt.legend()
191 plt.tight_layout()
192 plt.savefig("plots/corr_vs_horizon.png")
193 plt.close()
194 print("5.2A: saved plots/corr_vs_horizon.png")
195
196 print("\n5.2B")
197
198 def get_run(H):
199     Xtrain, ytrain_clean, _, _, Xtest, ytest_clean = build_datasets(N, W, H, sigma)
200     ytrain = add_outliers(ytrain_clean.copy(), p, a)
201
202     X = Xtrain.T
203     y = ytrain
204     XXT = X @ X.T
205     Xy = X @ y
206     w_mmse = np.linalg.solve(XXT, Xy)
207     yhat_mmse = Xtest @ w_mmse
208
209     svr = SVR(C=10.0, epsilon=0.01, kernel='linear')
210     svr.fit(Xtrain, ytrain)
211     yhat_svr = svr.predict(Xtest)
212
213     return ytest_clean, yhat_mmse, yhat_svr
214
215 Hs_scatter = [1, 5]
```

```

216
217 plt.figure(figsize=(10,8))
218 for i, H in enumerate(Hs_scatter, 1):
219     ytrue, yhat_m, yhat_s = get_run(H)
220     ymin = min(ytrue.min(), yhat_m.min(), yhat_s.min())
221     ymax = max(ytrue.max(), yhat_m.max(), yhat_s.max())
222     line = np.linspace(ymin, ymax, 100)
223
224     plt.subplot(2,2,2*i-1)
225     plt.scatter(ytrue, yhat_m, s=10, alpha=0.7)
226     plt.plot(line, line, 'r--')
227     plt.xlabel('True')
228     plt.ylabel('MMSE prediction')
229     plt.title(f'H = {H} (MMSE)')
230     plt.grid()
231
232     plt.subplot(2,2,2*i)
233     plt.scatter(ytrue, yhat_s, s=10, alpha=0.7)
234     plt.plot(line, line, 'r--')
235     plt.xlabel('True')
236     plt.ylabel('SVR prediction')
237     plt.title(f'H = {H} (SVR)')
238     plt.grid()
239
240 plt.tight_layout()
241 plt.savefig("plots/corr_scatter_H1_H5.png")
242 plt.close()
243 print("5.2B: saved plots/corr_scatter_H1_H5.png")
244
245 print("\n5.2D-F")
246
247 def svr_grid_search(H, epsilons, cs):
248     Xtrain, ytrain_clean, Xval, yval_clean, Xtest, ytest_clean = build_datasets(N, W, H, sigma)
249     ytrain = add_outliers(ytrain_clean.copy(), p, a)
250
251     corr_grid = np.zeros((len(epsilons), len(cs)))
252     for i, eps in enumerate(epsilons):
253         for j, C in enumerate(cs):
254             svr = SVR(C=C, epsilon=eps, kernel='linear')
255             svr.fit(Xtrain, ytrain)
256             yhat_val = svr.predict(Xval)
257             corr_grid[i, j] = corrcoef(yval_clean, yhat_val)
258
259     best_idx = np.unravel_index(np.argmax(corr_grid), corr_grid.shape)
260     best_eps = epsilons[best_idx[0]]
261     best_C = cs[best_idx[1]]
262
263     svr_best = SVR(C=best_C, epsilon=best_eps, kernel='linear')
264     svr_best.fit(Xtrain, ytrain)
265     yhat_test = svr_best.predict(Xtest)
266     corr_test = corrcoef(ytest_clean, yhat_test)
267
268     return Xtrain, ytrain_clean, Xval, yval_clean, Xtest, ytest_clean, corr_grid, best_eps, best_C, yhat_test
269

```

```
270 max_epsilons = 10
271 max_cs = 10
272 epsilons = np.logspace(-2, 0, max_epsilons)
273 cs      = np.logspace(-2, 2, max_cs)
274
275 pred_results = {}
276 contour_data = {}
277
278 for H in [1, 2]:
279     print(f"\nH = {H}")
280     (Xtrain, ytrain_clean, Xval, yval_clean,
281      Xtest, ytest_clean, corr_grid, best_eps, best_C,
282      yhat_test, corr_test) = svr_grid_search(H, epsilons, cs)
283
284     print(f"Best epsilon: {best_eps:.4f}, Best C: {best_C:.4f}, Test corr: {corr_test:.4f}")
285
286     contour_data[H] = corr_grid
287     pred_results[H] = (ytest_clean, yhat_test)
288
289     Xg, Yg = np.meshgrid(np.log10(cs), np.log10(epsilons))
290     plt.figure(figsize=(12,5))
291
292     for i, H in enumerate([1, 2], 1):
293         plt.subplot(1, 2, i)
294         cp = plt.contourf(Xg, Yg, contour_data[H], levels=20)
295         plt.colorbar(cp)
296         plt.xlabel(r'$\log_{10}(C)$')
297         plt.ylabel(r'$\log_{10}(\text{varepsilon})$')
298         plt.title(f'Correlation map (H={H})')
299         plt.grid(False)
300
301     plt.tight_layout()
302     plt.savefig("plots/contour_corr_combined.png")
303     plt.close()
304     print("Saved plots/contour_corr_combined.png")
305
306     print("\nSaving combined best-SVR test predictions...")
307
308     plt.figure(figsize=(12,5))
309
310     for i, H in enumerate([1, 2], 1):
311         ytrue, yhat_best = pred_results[H]
312         t = np.arange(len(ytrue))
313
314         plt.subplot(1, 2, i)
315         plt.plot(t, ytrue, label='Clean test', linewidth=2)
316         plt.plot(t, yhat_best, '--', label='Best SVR')
317         plt.xlabel("Time")
318         plt.ylabel("Signal")
319         plt.title(f"Best SVR Prediction (H={H})")
320         plt.grid()
321         plt.legend()
322
323     plt.tight_layout()
```



```
324 plt.savefig("plots/pred_best_SVR.png")
325 plt.close()
326
327 print("Saved plots/pred_best_SVR.png")
328
```